

UNIVERSIDADE ESTADUAL DO NORTE FLUMINENSE
LABORATÓRIO DE ENGENHARIA E EXPLORAÇÃO DE PETRÓLEO
CENTRO DE CIÊNCIA E TECNOLOGIA

PROJETO DE ENGENHARIA
DESENVOLVIMENTO DO SOFTWARE
PoreElasticCalculator

DISCIPLINA: Projeto de Software Aplicado à Engenharia

Matheus Bastos & Nicolau Prates
Prof. André Duarte Bueno

MACAÉ - RJ
Junho - 2025

Sumário

1	Concepção	6
1.1	Metodologia	6
1.2	Nome do Sistema/Produto	7
1.3	Especificação	7
1.4	Requisitos	7
1.4.1	Requisitos funcionais	7
1.4.2	Requisitos não funcionais	8
1.5	Casos de Uso	8
1.5.1	Diagrama de caso de uso geral	8
1.5.2	Diagrama de caso de uso específico	8
2	Elaboração	10
2.1	Análise de domínio	10
2.2	Formulação teórica	10
2.2.1	Modelo de Voigt-Reuss-Hill (VRH)	10
2.2.2	Limites de Hashin-Shtrikman (HS)	11
2.3	Identificação de pacotes – assuntos	11
2.4	Diagrama de pacotes – assuntos	12
3	AOO – Análise Orientada a Objeto	13
3.1	Diagramas de classes	13
3.1.1	Dicionário de classes	13
3.2	Diagrama de sequência – eventos e mensagens	18
3.2.1	Diagrama de sequência geral	18
3.2.2	Diagrama de sequência específico	19
3.3	Diagrama de comunicação – colaboração	19
3.4	Diagrama de estado	20
3.5	Diagrama de atividades	21
4	Projeto	23
4.1	Projeto do Sistema	23
4.2	Projeto Orientado a Objeto – POO	25
4.3	Diagrama de Componentes	27

4.4	Diagrama de Implantação	29
5	Lista das Características/<i>Features</i>	31
5.1	Lista de características «features»	31
	Referências Bibliográficas	33
A	Modelagem Matemática e Algoritmos de Cálculo	34
A.1	Modelos de Média Poroelástica	34
A.2	Algoritmo de Cálculo dos Módulos Elásticos	35
B	Código-Fonte Selecionado	36
B.1	Classe CSimulador	36
B.2	Classe CElasticCalculator	37
B.3	Classe CAmostra	37

Lista de Figuras

1.1	Metodologia utilizada no desenvolvimento do sistema	6
1.2	Diagrama de caso de uso – Caso de uso geral	8
1.3	Diagrama de caso de uso específico	9
2.1	Diagrama de Pacotes	12
3.1	Diagrama de classes	13
3.2	Diagrama de sequência geral	18
3.3	Diagrama de sequência específico	19
3.4	Diagrama de comunicação	20
3.5	Diagrama de máquina de estado	21
3.6	Diagrama de atividades	22
4.1	Diagrama de componentes	28
4.2	Diagrama de implantação	30

Lista de Tabelas

1.1	Caso de uso 1	8
-----	-------------------------	---

Capítulo 1

Concepção

Apresenta-se neste capítulo do projeto de engenharia a concepção, a especificação do sistema a ser modelado e desenvolvido.

1.1 Metodologia

A Figura 1.1 apresenta a metodologia a ser utilizada no desenvolvimento do sistema.

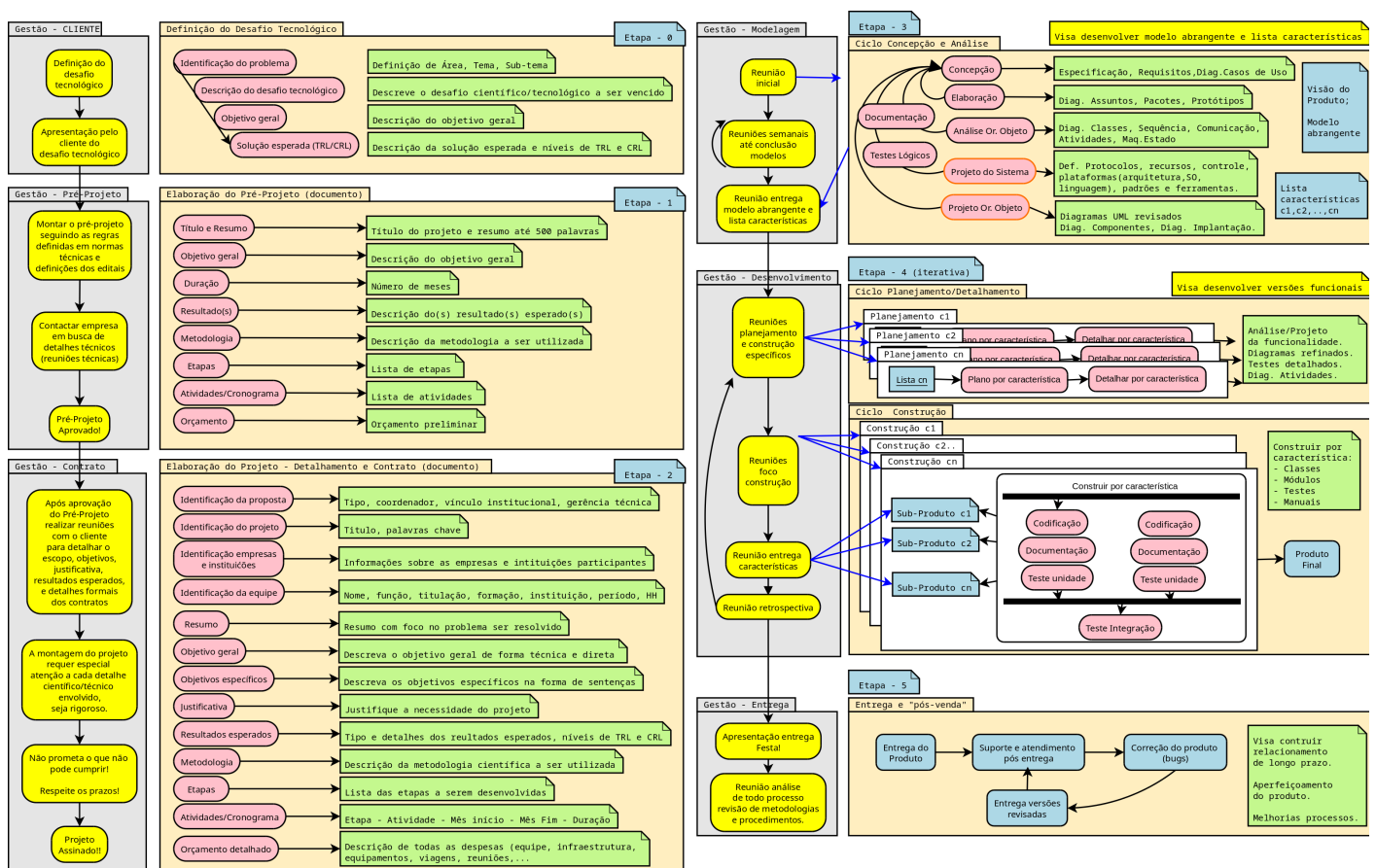


Figura 1.1: Metodologia utilizada no desenvolvimento do sistema

1.2 Nome do Sistema/Produto

Nome	PoroElasticCalculator
Componentes principais	MineralDatabase, ReservoirData, PECAApplication
Missão	Calcular propriedades elásticas de amostras de rocha

1.3 Especificação

O software PEC (PoroElasticCalculator) tem como objetivo estimar as propriedades elásticas efetivas de rochas sedimentares a partir de sua composição mineralógica, por meio da aplicação de modelos físicos consagrados na literatura de física das rochas.

O sistema visa auxiliar engenheiros, geofísicos e pesquisadores na avaliação de módulo de compressibilidade (K) e módulo de cisalhamento (G) de amostras naturais ou sintéticas.

1.4 Requisitos

Apresenta-se nesta seção os requisitos funcionais e não funcionais.

1.4.1 Requisitos funcionais

Apresenta-se a seguir os requisitos funcionais.

RF-01	O sistema deve conter uma base de dados dos módulos de cisalhamento e compressibilidade de vários minerais.
RF-02	O usuário deverá ter liberdade para colocar manualmente os valores de sua amostra ou carregar um arquivo de saída que já contenha estes dados.
RF-03	Deve permitir o carregamento de arquivos criados pelo software.
RF-04	Deve permitir a escolha das equações de Hashin-Strikeman ou Voig-Reus-Hill.
RF-05	O O usuário deve ter tal liberdade para escolher as porcentagens mineralógicas da sua amostra.
RF-06	O usuário poderá plotar seus resultados em um gráfico. O gráfico poderá ser salvo como imagem ou ter seus dados exportados como texto.

1.4.2 Requisitos não funcionais

RNF-01	Os cálculos devem ser feitos utilizando-se as equações de Hashin-Strikeman ou Voig-Reus-Hill.
RNF-02	O programa deverá ser multi-plataforma, podendo ser executado em <i>Windows</i> , <i>GNU/Linux</i> ou <i>MacOS</i> .

1.5 Casos de Uso

A Tabela 1.1 mostra a descrição de um caso de uso.

Tabela 1.1: Caso de uso 1

Nome do caso de uso:	Geral
Resumo/descrição:	Etapas a serem cumpridas pelo usuário
Etapas:	Criar a amostra, carregar os arquivos, gerar os crossplot, analisar os resultados.
Cenários alternativos:	O usuário escolhe os valores manualmente, podendo adicionar novos minerais

1.5.1 Diagrama de caso de uso geral

O diagrama de caso de uso geral da Figura 1.2 mostra o usuário acessando os sistemas de ajuda do software, calculando as propriedades elásticas e analisando os resultados.

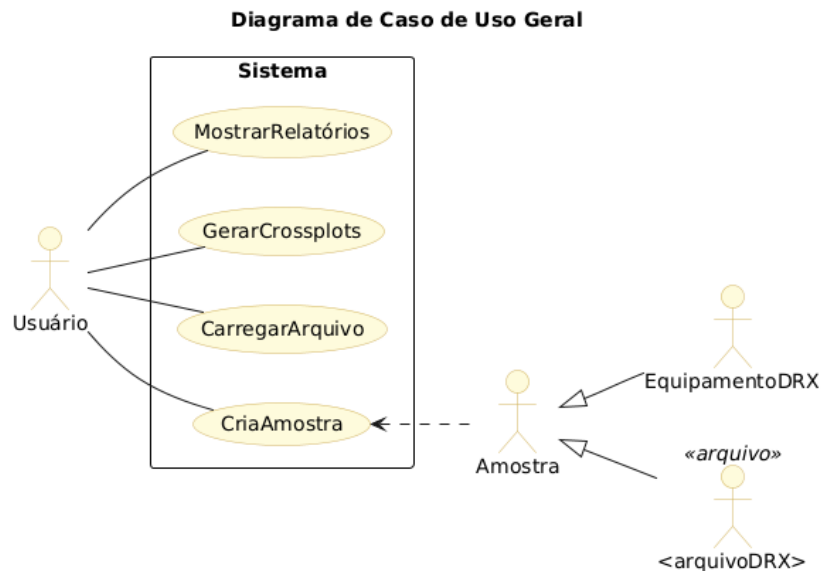


Figura 1.2: Diagrama de caso de uso – Caso de uso geral

1.5.2 Diagrama de caso de uso específico

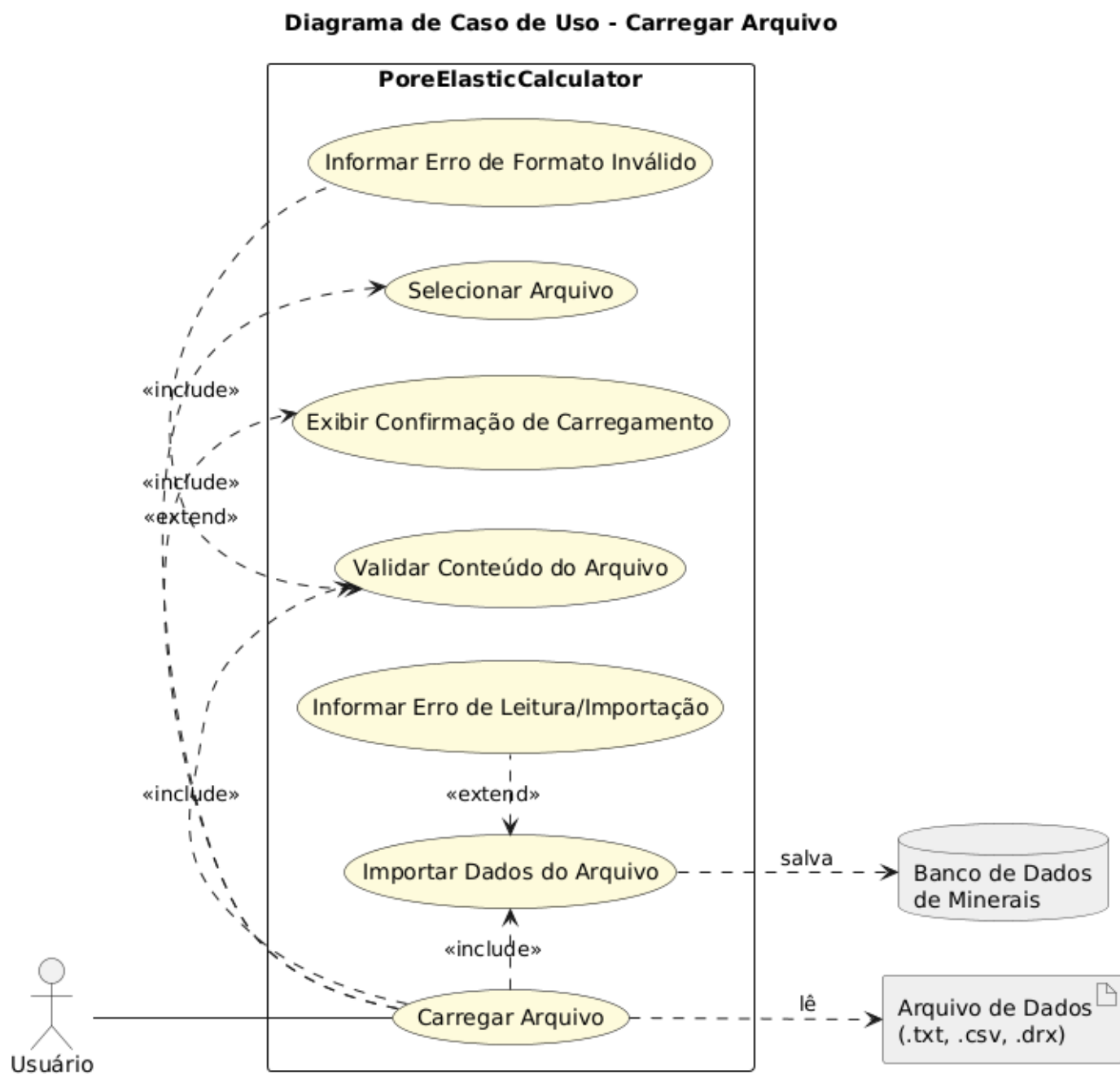


Figura 1.3: Diagrama de caso de uso específico

Capítulo 2

Elaboração

2.1 Análise de domínio

Após estudo dos requisitos/especificações do sistema, algumas entrevistas, estudos na biblioteca e disciplinas do curso foi possível identificar nosso domínio de trabalho:

- Área de aplicação: Engenharia de Petróleo, Petrofísica, Geomecânica
- Problema central: Determinar propriedades elásticas (módulo de cisalhamento e compressibilidade) de rochas com composições mineralógicas variadas, algo comum em formações sedimentares.

2.2 Formulação teórica

A determinação das propriedades elásticas efetivas de rochas com múltiplos constituintes minerais exige o uso de modelos de homogeneização que aproximem o comportamento macroscópico do meio composto. Neste trabalho, são utilizados os modelos de Voigt-Reuss-Hill (VRH) e os limites de Hashin-Shtrikman (HS) para a estimativa do módulo de compressibilidade K e do módulo de cisalhamento G da rocha.

2.2.1 Modelo de Voigt-Reuss-Hill (VRH)

O modelo de Voigt assume deformação uniforme entre os constituintes da mistura, resultando em uma média aritmética ponderada das propriedades dos minerais:

$$K_V = \sum_{i=1}^n f_i K_i \quad (2.1)$$

$$G_V = \sum_{i=1}^n f_i G_i \quad (2.2)$$

Já o modelo de Reuss considera tensão uniforme entre os constituintes, levando a uma média harmônica ponderada:

$$\frac{1}{K_R} = \sum_{i=1}^n \frac{f_i}{K_i} \quad (2.3)$$

$$\frac{1}{G_R} = \sum_{i=1}^n \frac{f_i}{G_i} \quad (2.4)$$

A média de Voigt-Reuss-Hill (VRH) é obtida como a média aritmética simples entre os valores de Voigt e Reuss:

$$K_{VRH} = \frac{K_V + K_R}{2} \quad (2.5)$$

$$G_{VRH} = \frac{G_V + G_R}{2} \quad (2.6)$$

2.2.2 Limites de Hashin-Shtrikman (HS)

Os limites de Hashin-Shtrikman oferecem uma estimativa teórica rigorosa dos valores mínimo e máximo possíveis para as propriedades elásticas de materiais isotrópicos compostos por duas ou mais fases. Supondo K_1 e G_1 como os módulos do mineral com maior rigidez e K_2 e G_2 como os módulos do mineral mais fraco, com frações volumétricas f_1 e f_2 , os limites são dados por:

$$K_{HS}^+ = K_2 + \frac{f_1}{1K_1 - K_2 + f_2K_2 + 43G_2} \quad (2.7)$$

$$G_{HS}^+ = G_2 + \frac{f_1}{1G_1 - G_2 + f_2G_2 + \zeta} \quad (2.8)$$

$$K_{HS}^- = K_1 + \frac{f_2}{1K_2 - K_1 + f_1K_1 + 43G_1} \quad (2.9)$$

$$G_{HS}^- = G_1 + \frac{f_2}{1G_2 - G_1 + f_1G_1 + \zeta'} \quad (2.10)$$

Onde:

$$\zeta = \frac{G_2(9K_2 + 8G_2)}{6(K_2 + 2G_2)} \quad ; \quad \zeta' = \frac{G_1(9K_1 + 8G_1)}{6(K_1 + 2G_1)} \quad (2.11)$$

Esses limites fornecem uma faixa possível para as propriedades efetivas, sendo úteis para validar os resultados obtidos com modelos mais simplificados como VRH.

2.3 Identificação de pacotes – assuntos

Este projeto fundamenta-se em um conjunto de pacotes teóricos e computacionais voltados para o estudo das propriedades elásticas de rochas com múltiplas composições mineralógicas. A seguir são descritos os principais pacotes envolvidos, com sua respectiva função no desenvolvimento do modelo.

- Rock Physics: Conjunto de teorias que conectam propriedades elásticas da rocha (como módulo de cisalhamento e compressibilidade) com suas características petrofísicas, como composição mi-

neralógica, porosidade e saturação. Este pacote fornece a base conceitual para a formulação dos modelos de mistura empregados neste trabalho.

- Mineral Physics: Ramo que estuda as propriedades físicas dos minerais puros, como o módulo de compressibilidade (K) e o módulo de cisalhamento (G). Esses valores são fundamentais como dados de entrada para os modelos de homogeneização. Os dados típicos são obtidos da literatura especializada.
- Método de homogeneização que fornece uma média das propriedades elásticas baseada em hipóteses de deformação (Voigt) e tensão (Reuss) uniformes. A média de Voigt-Reuss-Hill representa uma estimativa intermediária entre os extremos teóricos.
- Gnuplot, ferramenta de visualização utilizada para representar graficamente os resultados dos modelos, como comparação entre VRH e HS, análises de sensibilidade e tendências com variação mineralógica.
- Teoria dos meios efetivos: Pacote teórico que abrange todos os modelos de homogeneização para materiais compostos, como VRH, HS, DEM (Differential Effective Medium) e outros. Justifica conceitualmente o uso dos modelos implementados neste trabalho.

2.4 Diagrama de pacotes – assuntos

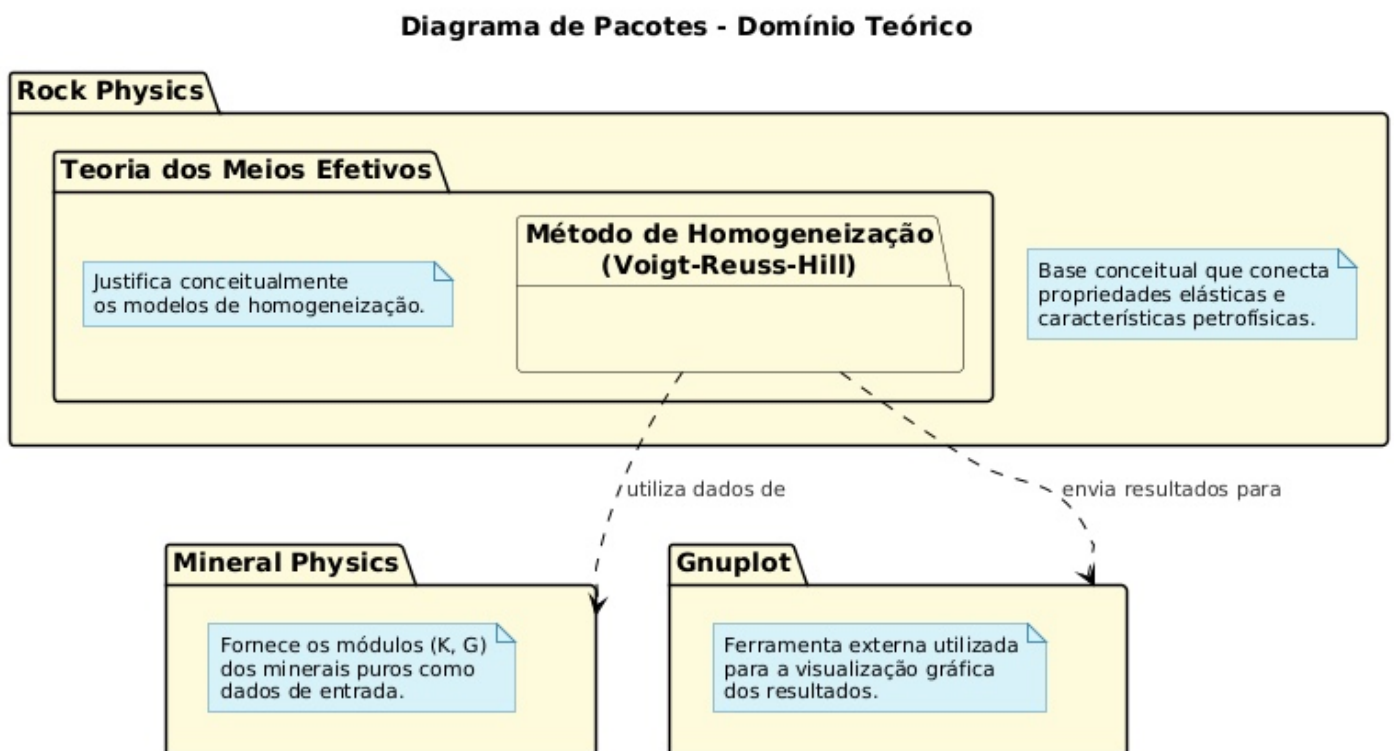


Figura 2.1: Diagrama de Pacotes

Capítulo 3

AOO – Análise Orientada a Objeto

3.1 Diagramas de classes

O diagrama de classes é apresentado na Figura 3.1.

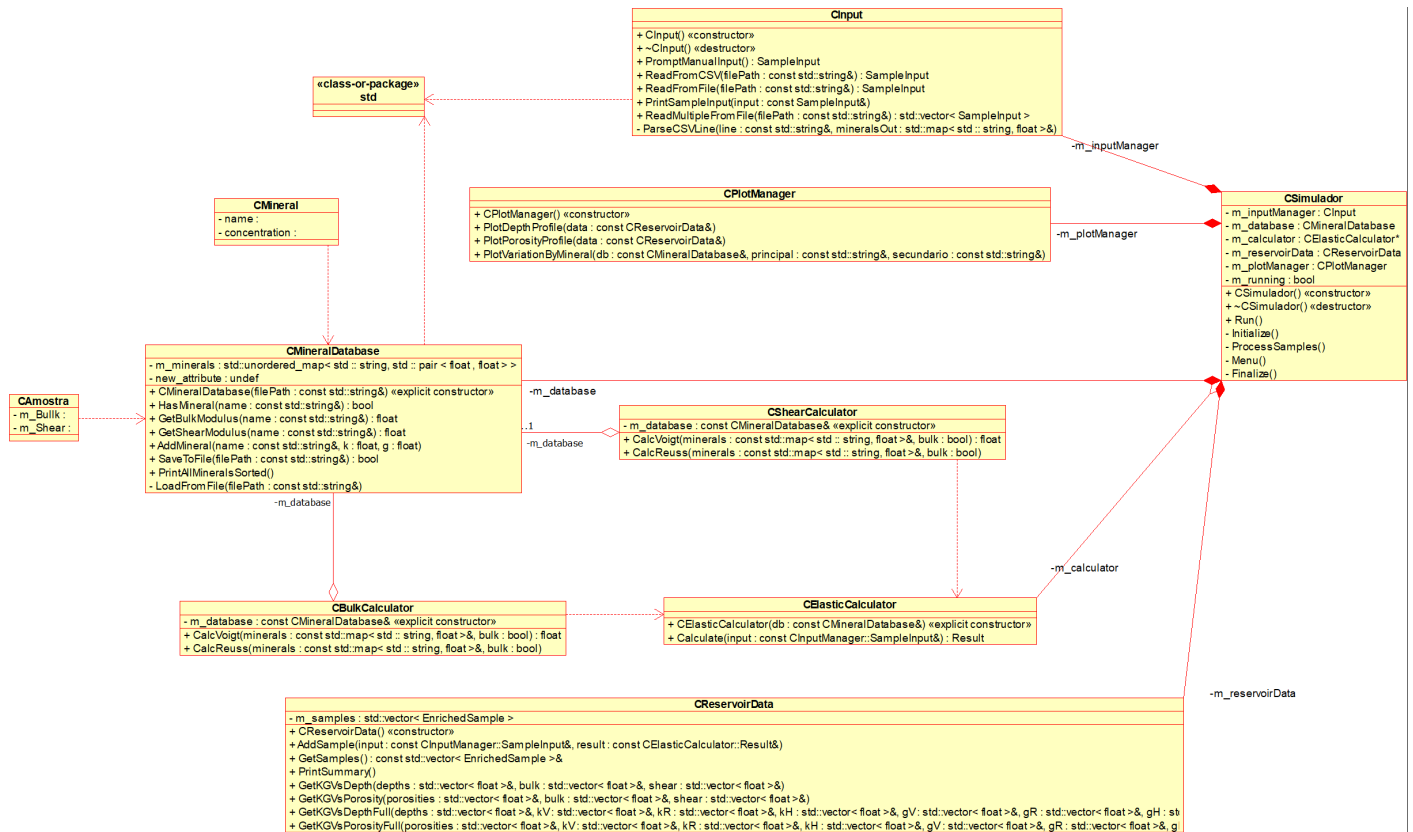


Figura 3.1: Diagrama de classes

3.1.1 Dicionário de classes

• CSimulator

Responsabilidade: Classe principal que orquestra a execução do programa. É responsável por inicializar todos os componentes, gerenciar o menu de interação com o usuário e controlar o fluxo de dados entre os módulos de entrada, cálculo e visualização.

Atributos:

- `-m_input`: `CInput` – Instância do gerenciador de entrada de dados.
- `-m_database`: `CMineralDatabase` – Instância do banco de dados de minerais.
- `-m_calculator`: `CElasticCalculator` – Instância do módulo de cálculo de propriedades elásticas.
- `-m_reservoirData`: `CReservoirData` – Instância do repositório de dados das amostras processadas.
- `-m_plotManager`: `CPlotManager` – Instância do gerenciador de visualização gráfica.

Métodos:

- `+CSimulator()` / `+~CSimulator()`: Construtor e destrutor da classe.

- `+Run()`: Inicia e gerencia o laço de execução principal da aplicação.
- `-Initialize()`: Prepara e aloca os recursos para os módulos do sistema.
- `-Menu()`: Exibe as opções disponíveis e captura a escolha do usuário.
- `-ProcessSamples()`: Dispara o fluxo de processamento para uma nova amostra.
- `-Finalize()`: Libera os recursos alocados antes de encerrar a aplicação.

• **CInput**

Responsabilidade: Gerencia a aquisição de dados das amostras, seja por meio de entrada manual do usuário no console ou pela leitura e interpretação de arquivos de dados formatados.

Métodos:

- `+CInput()` / `+~CInput()`: Construtor e destrutor da classe.

- `+PromptManualInput()`: Solicita interativamente os dados da amostra ao usuário.
- `+ReadFromCSV()`: Lê dados de um arquivo no formato CSV (não utilizado na versão final).
- `+ReadFromFile()`: Lê os dados de uma única amostra de um arquivo de texto.
- `+PrintSampleInput()`: Exibe os dados de entrada de uma amostra no console para verificação.
- `+ReadMultipleFromFile()`: Lê uma ou mais amostras de um arquivo de texto.
- `-ParseCSVLine()`: Interpreta uma única linha de um arquivo de dados para extrair a composição mineralógica.

• **CMineralDatabase**

Responsabilidade: Encapsula o acesso à base de dados de minerais. É responsável por carregar, salvar e fornecer as propriedades elásticas (módulo de compressibilidade e de cisalhamento) de cada mineral.

Atributos:

- `-m_minerals`: Estrutura de dados que armazena as propriedades dos minerais carregados em memória.

- `-m_databaseFilePath`: Caminho para o arquivo físico da base de dados.

Métodos:

- `+CMineralDatabase(filePath)`: Construtor que inicializa o objeto e carrega os dados do arquivo especificado.

- `+HasMineral(name)`: Verifica a existência de um mineral na base de dados.
- `+GetBulkModulus(name)`: Retorna o módulo de compressibilidade (K) de um mineral.
- `+GetShearModulus(name)`: Retorna o módulo de cisalhamento (G) de um mineral.
- `+AddMineral(name, k, g)`: Adiciona um novo mineral à base de dados em memória.
- `+SaveToFile(filePath)`: Salva o estado atual da base de dados em um arquivo.
- `+PrintAllMineralsSorted()`: Exibe todos os minerais da base de dados em ordem alfabética.
- `-LoadFromFile(filePath)`: Lógica interna para ler e interpretar o arquivo da base de dados.

• **CElasticCalculator**

Responsabilidade: Constitui o núcleo de cálculo do sistema. Implementa os modelos de homogeneização de Voigt e Reuss para estimar os módulos elásticos efetivos da matriz rochosa.

Atributos: – `-m_database: CMineralDatabase&` – Referência para a base de dados de minerais, utilizada para consultar as propriedades durante o cálculo.

Métodos: – `+CElasticCalculator(db)`: Construtor que recebe uma referência ao banco de dados de minerais.

- `+Calculate(input)`: Orquestra o cálculo para uma amostra, invocando os métodos de Voigt e Reuss e retornando os resultados.
- `-CalcVoigt(minerals, bulk)`: Implementa a média de Voigt para o módulo de compressibilidade ou de cisalhamento.
- `-CalcReuss(minerals, bulk)`: Implementa a média de Reuss para o módulo de compressibilidade ou de cisalhamento.

• **CReservoirData**

Responsabilidade: Atua como um repositório para as amostras processadas. Armazena os dados de entrada junto com os resultados calculados para uso posterior em relatórios e gráficos.

Atributos: – `-m_samples`: Vetor que armazena o conjunto de todas as amostras processadas durante a sessão.

Métodos: – `+CReservoirData()`: Construtor da classe.

- `+AddSample(input, result)`: Adiciona uma nova amostra e seus resultados calculados ao repositório.
- `+GetSamples()`: Retorna uma referência para o vetor completo de amostras.
- `+PrintSummary()`: Exibe no console um resumo com os principais resultados de todas as amostras.
- `+GetKGVvsDepth()`: Extrai e organiza os dados para plotagem de propriedades vs. profundidade.

- `+GetKGVvsPorosity()`: Extrai e organiza os dados para plotagem de propriedades vs. porosidade.
- `+GetKGVvsDepthFull()`: Retorna todos os limites (Voigt, Reuss, Hill) para os perfis de profundidade.
- `+GetKGVvsPorosityFull()`: Retorna todos os limites (Voigt, Reuss, Hill) para os perfis de porosidade.

• CPlotManager

Responsabilidade: Módulo dedicado à visualização de dados. Utiliza os resultados consolidados em `CReservoirData` para gerar e exibir gráficos, facilitando a análise e interpretação dos resultados.

Métodos: – `+CPlotManager()`: Construtor da classe.

- `+PlotDepthProfile(data)`: Gera o gráfico dos módulos elásticos em função da profundidade para as amostras fornecidas.
- `+PlotPorosityProfile(data)`: Gera o gráfico dos módulos elásticos em função da porosidade.
- `+PlotVariationByMineral(db, ...)`: Cria gráficos de sensibilidade à variação percentual entre dois minerais.

• CAmostra

Responsabilidade: Encapsula todos os dados pertinentes a uma única amostra de rocha. Funciona como uma estrutura de dados para transportar informações sobre a amostra através do sistema.

Atributos: – `-m_reservatorio`: Armazena o nome do reservatório de onde a amostra foi extraída.

- `-m_nome`: Identificador único da amostra.
- `-m_profundidade`: Profundidade (em metros) na qual a amostra foi coletada.
- `-m_porosidade`: Valor percentual da porosidade da rocha.
- `-m_minerais`: Estrutura de dados (ex.: mapa) que associa o nome de cada mineral à sua proporção na amostra.

Métodos: – `+CAmostra(...)`: Construtor que inicializa todos os atributos do objeto.

- `+NomeReservatorio()`: Retorna o nome do reservatório.
- `+NomeAmostra()`: Retorna o nome da amostra.
- `+Profundidade()`: Retorna o valor da profundidade.
- `+Porosidade()`: Retorna o valor da porosidade.
- `+Minerais()`: Retorna a coleção de minerais e suas respectivas proporções.

• CMineral

Responsabilidade: Representa a entidade de um mineral, contendo suas propriedades elásticas fundamentais.

Atributos:

- `-m_name`: O nome do mineral (ex.: Quartzo, Albita).
- `-m_bulkModulus`: Módulo de compressibilidade (K) do mineral, em GPa.
- `-m_shearModulus`: Módulo de cisalhamento (G) do mineral, em GPa.

Métodos:

- `+CMineral(name, k, g)`: Construtor que inicializa o objeto com os dados do mineral.
- `+Name()`: Retorna o nome do mineral.
- `+BulkModulus()`: Retorna o módulo de compressibilidade (K).
- `+ShearModulus()`: Retorna o módulo de cisalhamento (G).

• CShearCalculator

Responsabilidade: Especializada no cálculo do módulo de cisalhamento (G) efetivo de uma amostra de rocha, com base em sua composição mineralógica.

Atributos:

- `-m_db`: Referência à `CMineralDatabase`, utilizada para consultar propriedades de cisalhamento dos minerais individuais.

Métodos:

- `+CShearCalculator(db)`: Construtor que recebe a base de dados de minerais.
- `+CalculateVoigt(amostra)`: Calcula o módulo de cisalhamento efetivo utilizando a média de Voigt (limite superior).
- `+CalculateReuss(amostra)`: Calcula o módulo de cisalhamento efetivo utilizando a média de Reuss (limite inferior).
- `+CalculateHill(amostra)`: Calcula o módulo de cisalhamento efetivo utilizando a média de Hill (média entre Voigt e Reuss).

• CBulkCalculator

Responsabilidade: Especializada no cálculo do módulo de compressibilidade (K) efetivo de uma amostra de rocha, com base em sua composição mineralógica.

Atributos:

- `-m_db`: Referência à `CMineralDatabase`, utilizada para consultar propriedades de compressibilidade dos minerais individuais.

Métodos:

- `+CBulkCalculator(db)`: Construtor que recebe a base de dados de minerais.
- `+CalculateVoigt(amostra)`: Calcula o módulo de compressibilidade efetivo utilizando a média de Voigt.
- `+CalculateReuss(amostra)`: Calcula o módulo de compressibilidade efetivo utilizando a média de Reuss.
- `+CalculateHill(amostra)`: Calcula o módulo de compressibilidade efetivo utilizando a média de Hill.

3.2 Diagrama de sequência – eventos e mensagens

3.2.1 Diagrama de sequência geral

Veja o diagrama de sequência na Figura 3.2.

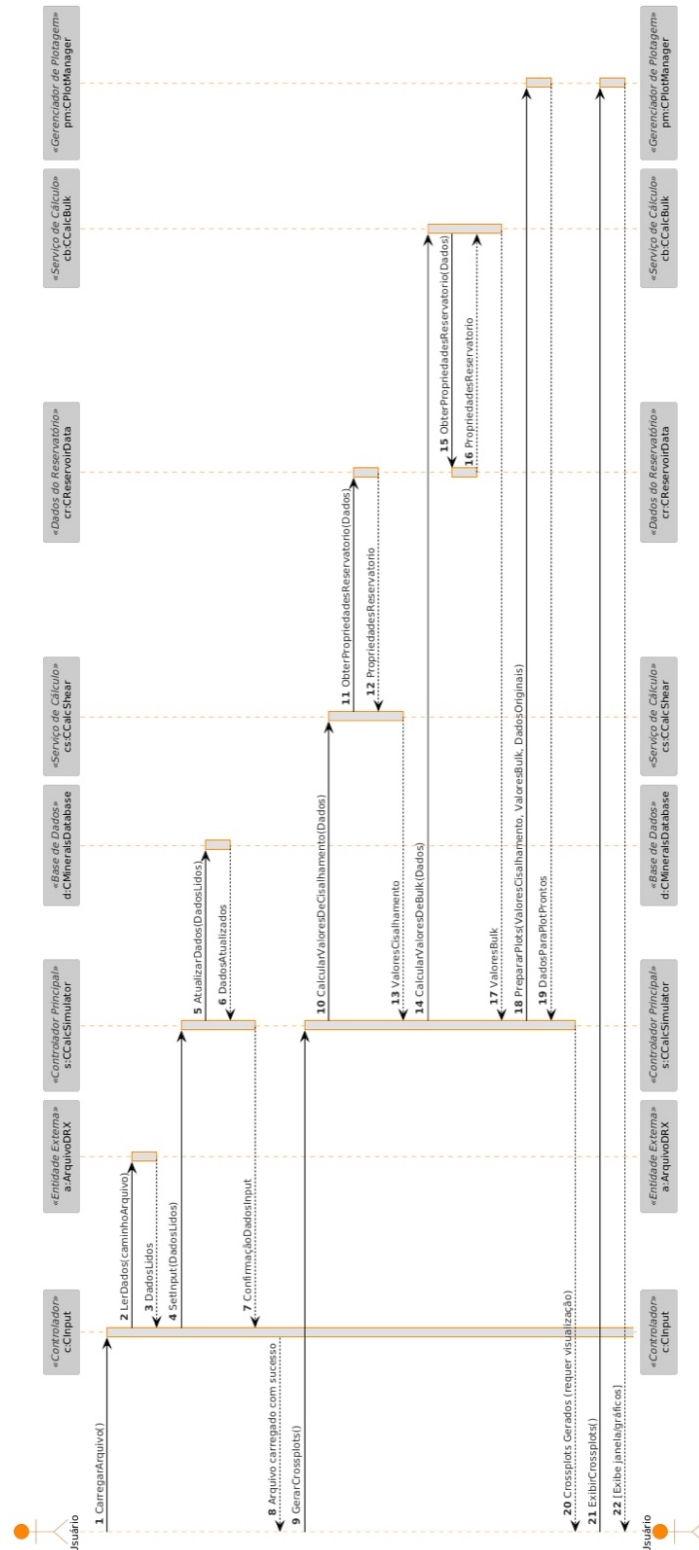


Figura 3.2: Diagrama de sequência geral

3.2.2 Diagrama de sequência específico

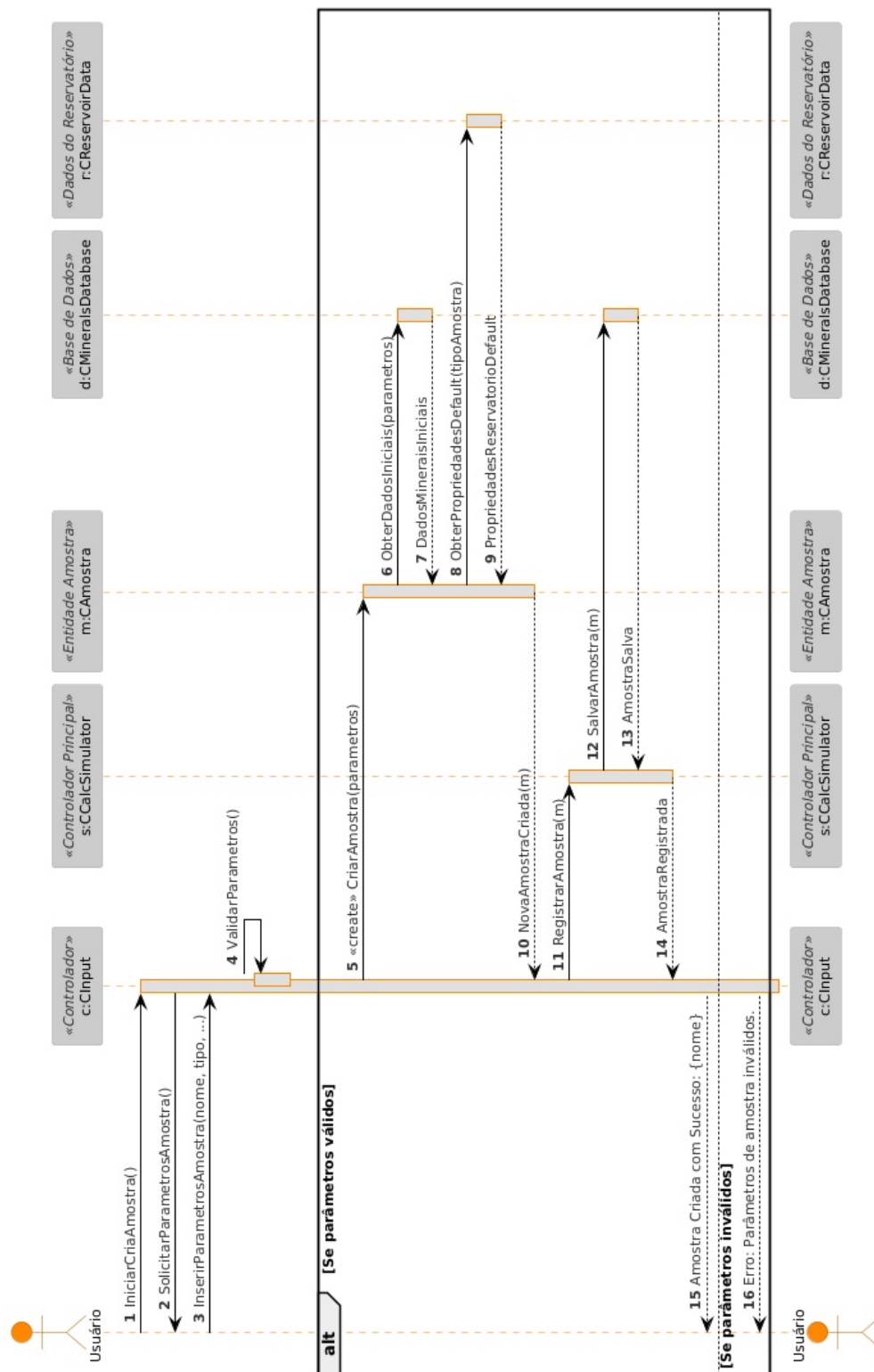


Figura 3.3: Diagrama de sequência específico

3.3 Diagrama de comunicação – colaboração

No diagrama de comunicação o foco é a interação e a troca de mensagens e dados entre os objetos.

Veja na Figura 3.3 o diagrama de comunicação mostrando a sequência de interação, onde a aplicação solicita leitura do arquivo da amostra para dar prosseguimento com o cálculo das propriedades, onde busca os dados do banco de dados. A aplicação armazena os dados e gera os resultados, e por fim solicita a geração dos gráficos de perfil.

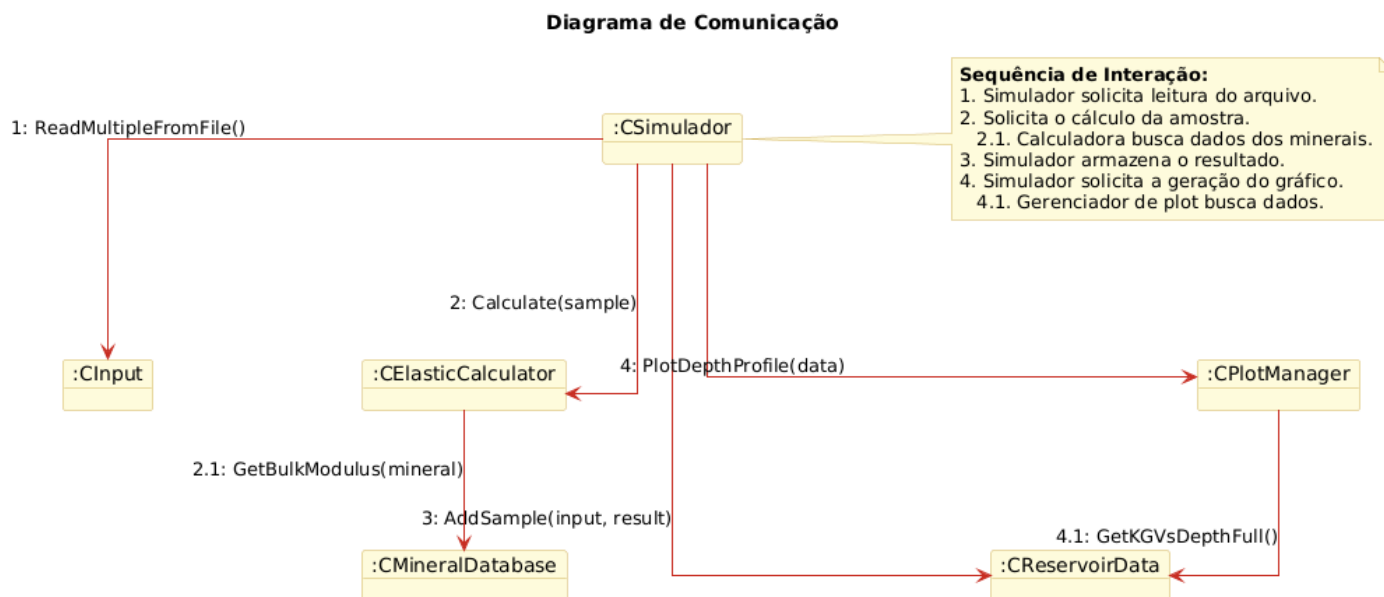


Figura 3.4: Diagrama de comunicação

3.4 Diagrama de estado

Veja na Figura 3.5 o diagrama de máquina de estado para o objeto CSimulador

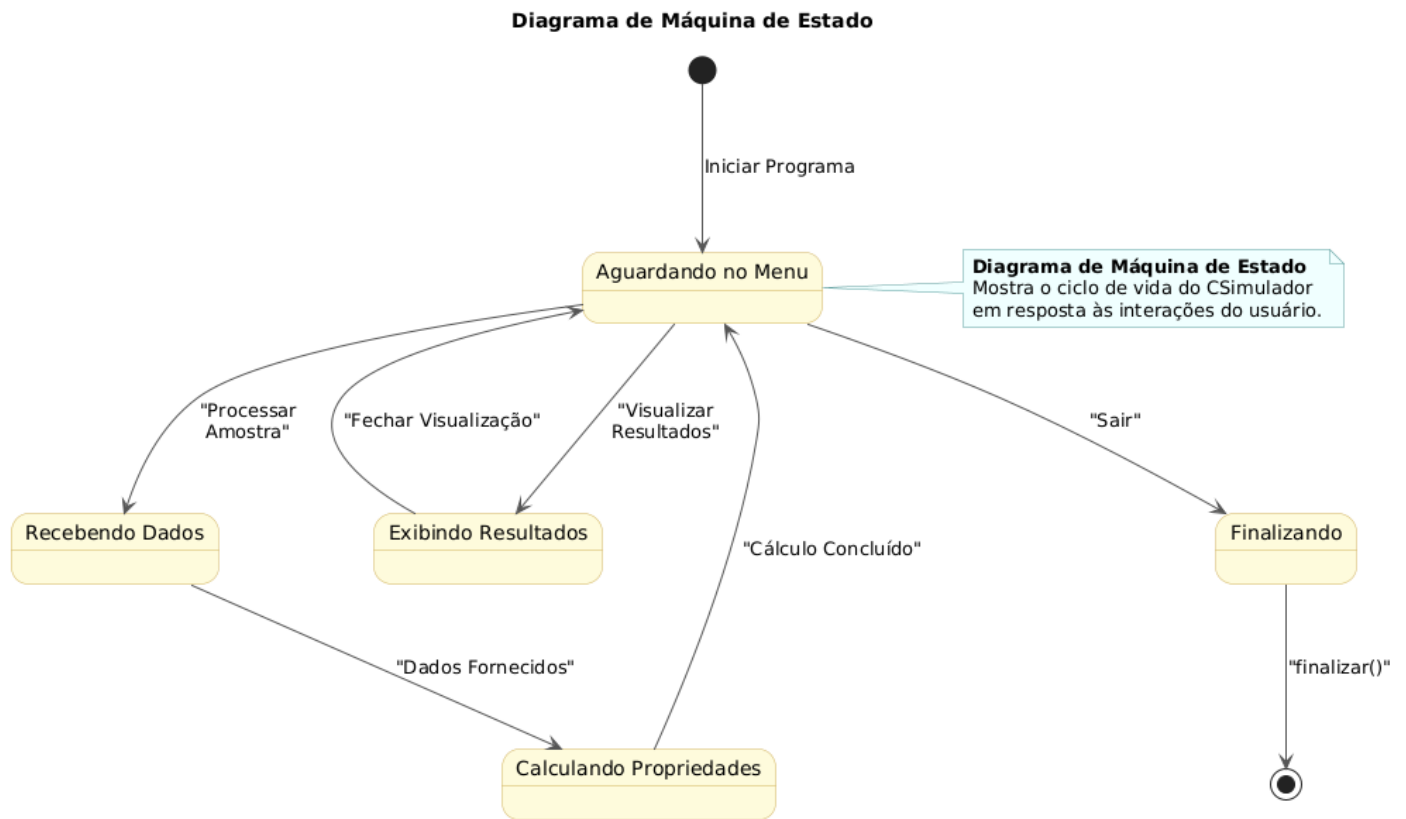


Figura 3.5: Diagrama de máquina de estado

3.5 Diagrama de atividades

Veja na Figura 3.6 o diagrama de atividades.

O processo tem início no Nó Inicial e transita para um laço de repetição, o qual representa o ciclo de vida interativo do menu principal e se mantém ativo enquanto a condição de controle `m_running` for verdadeira. Dentro deste laço, a primeira Atividade consiste em "Exibir Opções do Menu" no console, seguida imediatamente pela Atividade "Ler a escolha do usuário".

Ato contínuo, o fluxo chega a um Nó de Decisão, implementado como uma estrutura `switch`, que direciona o controle com base na opção fornecida. Se o usuário escolher as opções '1' ("Inserir nova amostra manualmente") ou '2' ("Carregar amostra de arquivo"), o fluxo é direcionado para as respectivas atividades de entrada de dados, que culminam no cálculo e armazenamento de novas amostras. As opções '3', '4' e '5' conduzem a atividades de visualização de dados, como "Ver resumo das amostras" ou a geração de gráficos específicos através da invocação dos métodos correspondentes.

Caso a opção selecionada seja '6' ("Sair"), a atividade `m_running = false` é executada, o que garante a terminação do laço de repetição na sua próxima iteração. Qualquer outra entrada de usuário que não corresponda às opções válidas levará à atividade "Exibir 'Opção inválida!'", e o controle retornará ao início do laço, aguardando uma nova entrada.

Uma vez que a condição de saída do laço é satisfeita, o fluxo de controle avança para a atividade final "Chamar Finalize()", que executa os procedimentos de encerramento do sistema. O processo culmina, então, no Nó Final, representando a terminação completa e ordenada da aplicação.

Diagrama de Atividades - CSimulador::Menu()

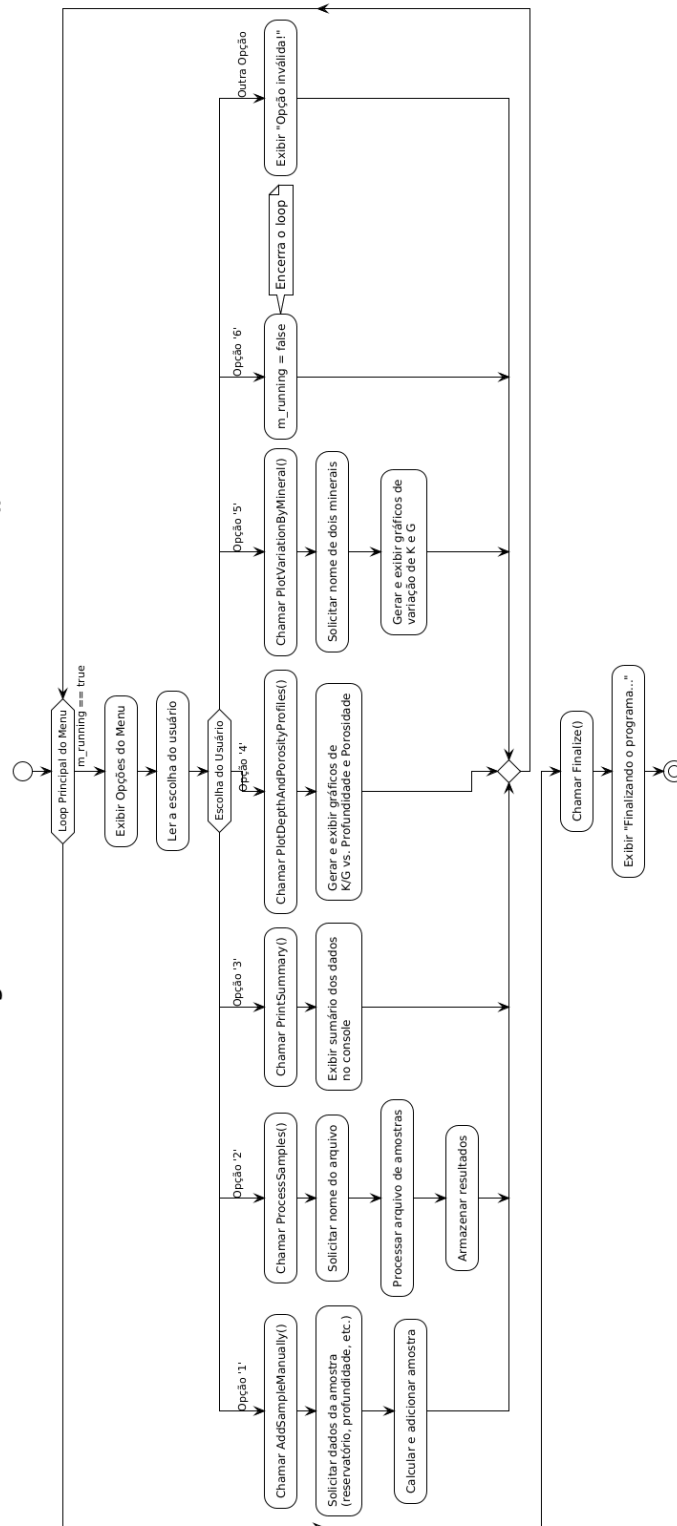


Figura 3.6: Diagrama de atividades

Capítulo 4

Projeto

Neste capítulo do projeto de engenharia veremos questões associadas ao projeto do sistema, incluindo protocolos, recursos, plataformas suportadas, implicações nos diagramas feitos anteriormente, diagramas de componentes e implantação. Na segunda parte revisamos os diagramas levando em conta as decisões do projeto do sistema.

4.1 Projeto do Sistema

Depois da análise orientada a objeto desenvolve-se o projeto do sistema, qual envolve etapas como a definição dos protocolos, da interface API, o uso de recursos, a subdivisão do sistema em subsistemas, a alocação dos subsistemas ao hardware e a seleção das estruturas de controle, a seleção das plataformas do sistema, das bibliotecas externas, dos padrões de projeto, além da tomada de decisões conceituais e políticas que formam a infraestrutura do projeto.

Deve-se definir padrões de documentação, padrões para o nome das classes, padrões de retorno e de parâmetros em métodos, características da interface do usuário e características de desempenho.

O projeto do sistema é a estratégia de alto nível para resolver o problema e elaborar uma solução.

1. Protocolos

- O sistema é concebido para ser uma aplicação desktop autônoma, com capacidade de importar dados e exportar resultados. A comunicação com dispositivos externos (como sensores laboratoriais, espectrômetros, etc.) não será necessária nesta primeira versão. No entanto, o software deverá:
 - Ler arquivos de entrada com composições mineralógicas em formatos abertos, preferencialmente .csv e .xlsx
 - Exportar os resultados dos cálculos em formatos igualmente abertos em .txt (relatórios simples); .csv (planilhas com resultados); .png (gráficos e visualizações)
- O sistema será implementado seguindo o paradigma de programação orientada a objetos, sendo os principais componentes do sistema representados como objetos com responsabilidades bem definidas.

A comunicação entre objetos seguirá os princípios do acoplamento fraco e alta coesão, utilizando:

- Chamadas diretas de métodos
- Passagem de parâmetros por instância (composição e agregação)
- Padrão de projeto MVC (Model-View-Controller), separando: Modelos físicos e matemáticos (Model); Interface com o usuário (View); Lógica de controle e fluxo de dados (Controller)
- O sistema utilizará os seguintes formatos abertos para arquivos de entrada:
 - .csv: para dados de composição mineralógica, módulos dos minerais, nomes de fases, etc.
 - .json: (opcional) para configurações de execução (modelo a usar, limites, parâmetros fixos)
- O sistema utilizará os seguintes formatos abertos para arquivos de saída:
 - .csv: resultados tabulares
 - .txt: logs e relatórios legíveis
 - .png / .pdf: gráficos comparativos
- Recursos
 - O sistema utiliza um banco de dados embutido contendo as propriedades elásticas conhecidas dos minerais (módulo de compressibilidade K, módulo de cisalhamento G, profundidade da amostra, porosidade). Este banco de dados permite a automação da busca de propriedades físicas a partir da identificação do nome do mineral, eliminando a necessidade de inserção manual dos parâmetros. Exemplo de estrutura da tabela minerais:

nome	k	g
albita	55	29.5
anidrita	55	30
anortita	84	40
aragonita	47	38.5
⋮	⋮	⋮

- Plataformas
 - * Padrão de Nomes
 - Classes: Utilizam o prefixo C seguido pelo formato PascalCase (ex: CInput, CSimulador).
 - Métodos e Funções: Adotam o formato PascalCase (ex: ParseSampleBlock, GetBulkModulus).
 - Variáveis Membro (Atributos): São prefixadas com m e seguem o formato camelCase (ex: mreservatorio, mdatabase).
- Padrão de retorno e parâmetros, que devem retornar:

- * Retorno de Dados: Os métodos que realizam cálculos ou processamentos retornam, preferencialmente, objetos que encapsulam os resultados, como instâncias de classes (CAmostra, Result) ou coleções padrão (std::vector).
- * Tratamento de Erros: Em caso de falhas, como a não localização de um mineral na base de dados, o sistema adota a estratégia de notificar o erro através de mensagens no fluxo de erro padrão (std::cerr), permitindo que a execução prossiga de forma controlada. A implementação de um sistema de exceções customizadas é vislumbrada como um aprimoramento futuro para um tratamento de erros ainda mais robusto.

4.2 Projeto Orientado a Objeto – POO

O projeto orientado a objeto é a etapa posterior ao projeto do sistema. Baseia-se na análise, mas considera as decisões do projeto do sistema. Acrescenta à análise desenvolvida as características da plataforma escolhida (hardware, sistema operacional e linguagem de programação). Passa pelo maior detalhamento do funcionamento do software, acrescentando atributos e métodos que envolvem a solução de problemas específicos não identificados durante a análise.

Envolve a otimização da estrutura de dados e dos algoritmos, a minimização do tempo de execução, de memória e de custos. Existe um desvio de ênfase para os conceitos da plataforma selecionada.

Efeitos do projeto no modelo estrutural

- Neste projeto, o modelo estrutural foi enriquecido com a inclusão de componentes da Biblioteca Padrão (STL) do C++. Pacotes como <vector>, <string>, <map>, <fstream> e <sstream> foram fundamentais para a implementação das estruturas de dados e das operações de entrada/saída. Adicionalmente, a decisão de utilizar a ferramenta externa Gnuplot para a visualização de dados introduziu uma dependência de sistema, refletida na arquitetura através da classe CPlotManager, cuja responsabilidade é fazer a interface com este subsistema.
- A escolha da linguagem C++ e de suas bibliotecas padrão resultou na concretização de classes e associações. Por exemplo, a classe CInput utiliza std::ifstream para a leitura de arquivos, e a CPlotManager emprega std::ofstream para gerar os scripts .gp e os arquivos de dados .dat. A classe CAmostra utiliza std::map<std::string, float> para materializar a associação entre a amostra e sua composição mineralógica, uma decisão de projeto que garante eficiência na busca por minerais.
- A plataforma escolhida impõe a dependência de um compilador C++11 (ou superior) para garantir a compatibilidade com os recursos da STL utilizados. A interação com o Gnuplot estabelece a restrição de que este software deve estar instalado e acessível no PATH do sistema operacional para que a funcionalidade de plotagem de gráficos opere corretamente.

Efeitos do projeto no modelo dinâmico

- A escolha da plataforma C++ e da biblioteca padrão influenciou diretamente os diagramas de sequência. Uma operação como "processar amostras" se desdobra em uma sequência detalhada: o objeto `CSimulador` invoca `CInput::ReadFile`, que internamente utiliza `std::ifstream` para ler o arquivo de entrada. Para cada bloco de amostra lido, `CSimulador` instancia `CAmostra` e invoca `CElasticCalculator::Calculate`. Este, por sua vez, interage com `CMineralDatabase` para obter as propriedades de cada mineral antes de retornar o resultado. Este nível de detalhe é uma consequência direta das decisões de projeto.
- A classe `CSimulador` implementa um laço de execução em seu método `Run()`, que opera com base em um menu interativo gerenciado pelo método `Menu()`. Este comportamento pode ser modelado por um diagrama de máquina de estados, onde cada opção do menu representa uma transição para um estado diferente da aplicação (ex: "Carregando Dados", "Calculando Propriedades", "Gerando Gráficos", "Finalizado"). A análise inicial poderia não ter previsto este nível de interação, que foi adicionado durante o projeto para melhorar a usabilidade da ferramenta.

Efeitos do projeto nos atributos

- Neste projeto, a transição da análise para o projeto introduziu atributos específicos da implementação. A classe `CMineralDatabase`, por exemplo, possui o atributo `m_databaseFilePath`, que armazena o caminho para o arquivo `.csv`, um detalhe de implementação para o acesso ao disco. Da mesma forma, as classes `CElasticCalculator`, `CBulkCalculator` e `CShearCalculator` contêm uma referência constante à base de dados (`m_db`), um ponteiro de implementação que otimiza o acesso aos dados dos minerais sem a necessidade de recarregá-los.

Efeitos do projeto nos métodos

- Os métodos da classe `CPlotManager` são um claro exemplo do impacto da plataforma escolhida. A necessidade de interagir com o Gnuplot levou à criação de métodos como `ExportCurve`, que realiza acesso a disco para criar arquivos `.dat`, e `GeneratePlotScript`, que não apenas escreve um script em um arquivo, mas também executa um comando de sistema (`system()`) para invocar um processo externo.
- No sistema, observa-se a distinção entre métodos políticos e de processamento. O método `CSimulador::Menu()` é eminentemente político, orquestrando o fluxo de controle da aplicação com base na entrada do usuário. Em contrapartida, os métodos `CalculateVoigt`, `CalculateReuss` e `CalculateHill` nas classes `CBulkCalculator` e `CShearCalculator` são puramente de processamento, focados na execução otimizada de fórmulas matemáticas.
- Os métodos implementados respondem diretamente às responsabilidades de suas classes. A responsabilidade da `CReservoirData` é armazenar e fornecer dados agregados. Seus métodos,

como `GetKGVsDepth` e `GetKGVsPorosity`, cumprem precisamente esta função, preparando os dados no formato exato que a `CPlotManager` necessita.

Efeitos do projeto nas heranças

- Neste projeto, optou-se pela **composição em detrimento da herança**. Não há hierarquias de herança, uma decisão de projeto deliberada que favorece a flexibilidade e um acoplamento mais fraco. Por exemplo, a classe `CElasticCalculator` contém instâncias de `CBulkCalculator` e `CShearCalculator`, em vez de herdar delas. Esta abordagem simplifica o design, pois as responsabilidades de cálculo são delegadas a objetos especializados.

Efeitos do projeto nas associações

- As associações foram implementadas utilizando os contêineres da STL. A associação "um-para-muitos" entre `CReservoirData` e `CAmostra` é materializada por um `std::vector<Enriched Sample>`. Esta escolha permite o armazenamento ordenado e o acesso indexado às amostras processadas.
- A associação entre uma `CAmostra` e seus minerais é implementada com um `std::map<std::string, float>`. Este "dicionário" é ideal, pois usa o nome do mineral como chave para um acesso rápido (complexidade logarítmica) à sua proporção, além de garantir a unicidade.

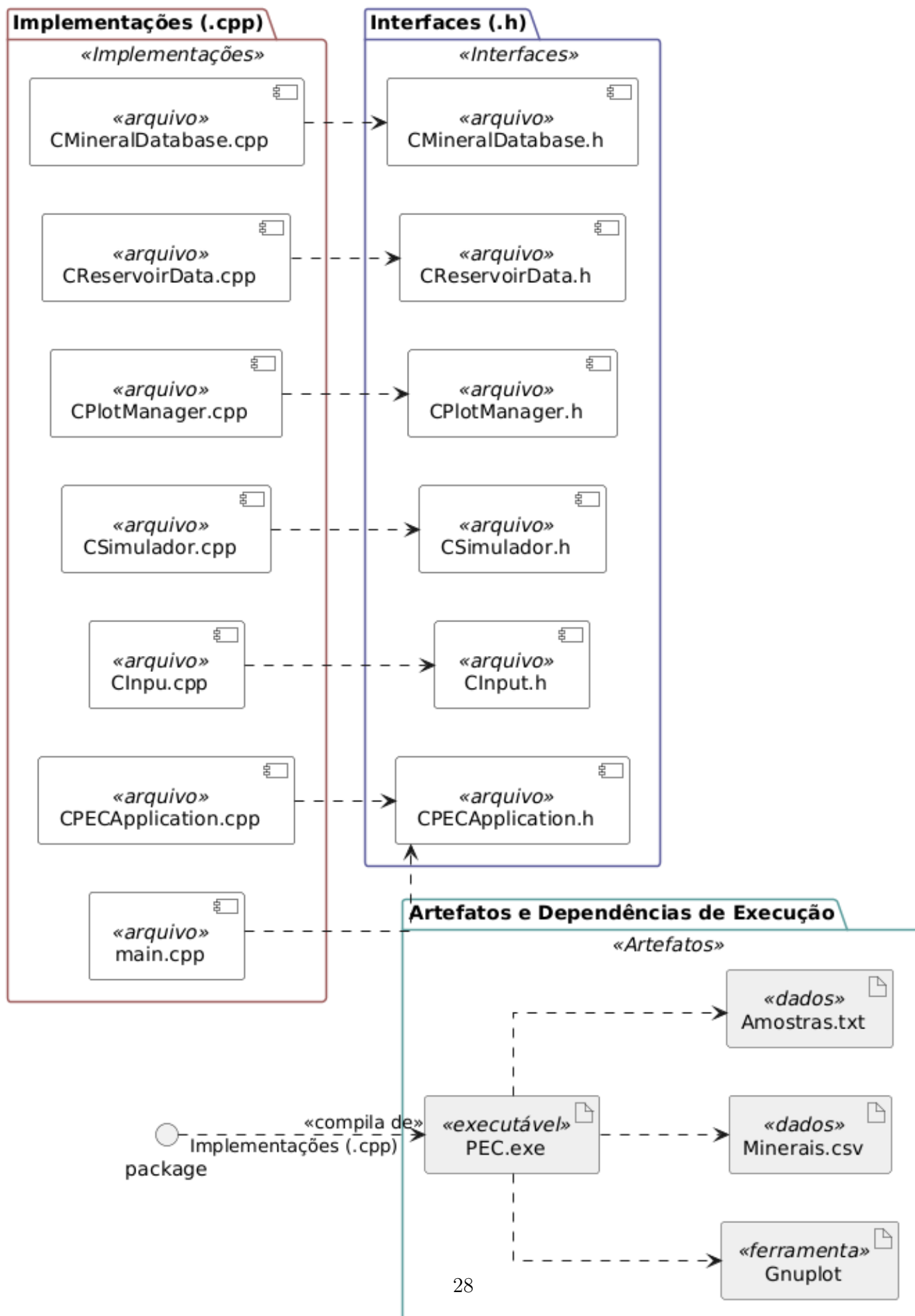
Efeitos do projeto nas otimizações

- A principal otimização no design de dados é o uso de `std::map` na classe `CMineralDatabase` para armazenar os minerais. Isso garante que a busca por um mineral durante os cálculos seja realizada em tempo logarítmico, o que é significativamente mais eficiente do que uma busca linear.
- O sistema opera de forma sequencial. Uma futura otimização poderia envolver a utilização de concorrência (`std::thread` ou `std::async`) para processar múltiplas amostras em paralelo, otimizando o uso de processadores multi-core.
- O design plano e com baixo acoplamento, onde `CSimulador` mantém referências diretas aos subsistemas, já previne caminhos de acesso longos, tornando desnecessária a adição de associações extras para fins de otimização.

4.3 Diagrama de Componentes

Segue o diagrama de componentes:

Diagrama de Componentes



4.4 Diagrama de Implantação

O diagrama de implantação é um diagrama de alto nível que inclui relações entre o sistema e o hardware e que se preocupa com os aspectos da arquitetura computacional escolhida. Seu enfoque é o hardware, a configuração dos nós em tempo de execução.

O diagrama de implantação deve incluir os elementos necessários para que o sistema seja colocado em funcionamento: computador, periféricos, processadores, dispositivos, nós, relacionamentos de dependência, associação, componentes, subsistemas, restrições e notas.

Neste projeto, o sistema de cálculo poroelástico foi concebido para ser executado em um ambiente de workstation local, como um computador pessoal ou um servidor de menor porte, sem a necessidade de uma infraestrutura de cluster distribuído. A interação com o usuário ocorre através da interface de linha de comando, e a geração de gráficos depende da instalação local do Gnuplot.

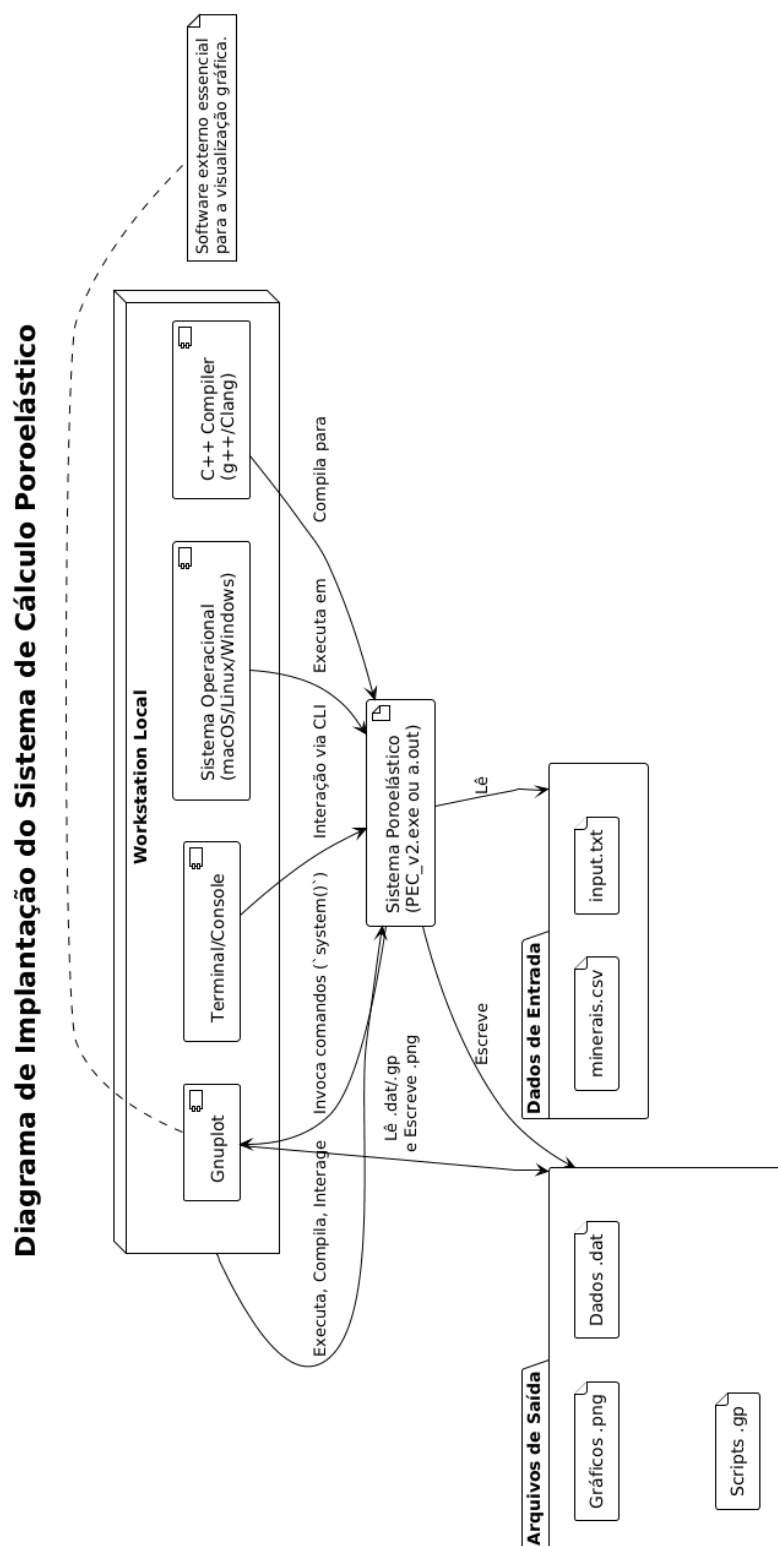


Figura 4.2: Diagrama de implantação

Capítulo 5

Lista das Características/*Features*

Neste capítulo, lista-se as características do sistema desenvolvido, organizadas de forma incremental de acordo com as versões do projeto. Esta abordagem visa a demonstrar a evolução da arquitetura e da funcionalidade do software, desde sua concepção inicial até a versão final apresentada neste trabalho.

5.1 Lista de características «features»

Após a análise dos requisitos e considerando a natureza acadêmica e didática do projeto, a implementação foi dividida em duas versões principais, culminando na seguinte lista de funcionalidades:

Versão 1.0 - Núcleo de Cálculo Poroelástico

- **Cálculo dos Módulos Elásticos:** O sistema deve ser capaz de calcular os módulos de compressibilidade (K) e de cisalhamento (G) efetivos de um compósito mineral utilizando as médias de Voigt, Reuss e Hill.

Classes implementadas: CElasticCalculator, CBulkCalculator, CShearCalculator.

- **Gerenciamento da Base de Dados de Minerais:** O sistema deve carregar e gerenciar uma base de dados de minerais a partir de um arquivo externo (.csv), permitindo a consulta das propriedades elásticas de cada mineral pelo nome.

Classe implementada: CMineralDatabase.

- **Leitura de Arquivo de Entrada Simplificado:** O sistema deve ser capaz de ler um arquivo de texto com a definição de uma única amostra de rocha, incluindo sua composição mineralógica e porosidade.

Classe implementada: CInput.

- **Visualização Gráfica:** O sistema deve gerar gráficos que correlacionem as propriedades elásticas calculadas com a variação da composição entre dois minerais.

Classe implementada: CPlotManager.

- **Orquestração Central:** O sistema deve possuir uma classe principal para coordenar o fluxo de execução, desde a leitura dos dados até a apresentação dos resultados.

Classe implementada: CSimulador.

Versão 2.0 - Arquitetura Refinada e Análise Multi-Amostra

- **Abstração da Entidade Amostra:** Para aprimorar a coesão e o encapsulamento, o conceito de "amostra de rocha" foi abstraído em uma classe própria, responsável por conter todos os seus dados característicos.

Classe implementada: CAmostra.

- **Abstração da Entidade Mineral:** De forma análoga, a entidade "mineral" foi encapsulada em sua própria classe, refinando a estrutura da base de dados e tornando o código mais claro e orientado a objeto.

Classe implementada: CMineral.

- **Processamento de Múltiplas Amostras:** O sistema foi aprimorado para ler e processar arquivos de entrada contendo a definição de múltiplas amostras de diferentes profundidades e porosidades.

Evolução da classe: CInput.

- **Gerenciamento de Dados de Reservatório:** Foi introduzida uma classe para agregar e gerenciar o conjunto de todas as amostras processadas e seus respectivos resultados, facilitando a análise consolidada.

Classe implementada: CReservoirData.

- **Geração de Perfis de Profundidade e Porosidade:** A capacidade de visualização foi expandida para permitir a plotagem de perfis contínuos, mostrando a variação dos módulos elásticos (K e G) em função da profundidade ou da porosidade para o conjunto de amostras analisadas.

Evolução da classe: CPlotManager.

Referências Bibliográficas

- [1] MAVKO, G.; MUKERJI, T.; DVORKIN, J. **The Rock Physics Handbook: Tools for Seismic Analysis of Porous Media**. 3rd ed. Cambridge: Cambridge University Press, 2020.
- [2] BUENO, A. D. **Programação Orientada a Objeto com C++**. 1^a ed. São Paulo: Novatec Editora, 2003.
- [3] HILL, R. The Elastic Behaviour of a Crystalline Aggregate. **Proceedings of the Physical Society. Section A**, v. 65, n. 5, p. 349-354, 1952.
- [4] GASSMANN, F. Über die Elastizität poröser Medien. **Vierteljahrsschrift der Naturforschenden Gesellschaft in Zürich**, v. 96, p. 1-23, 1951.
- [5] BIOT, M. A. Theory of Propagation of Elastic Waves in a Fluid-Saturated Porous Solid. I. Low-Frequency Range. **The Journal of the Acoustical Society of America**, v. 28, n. 2, p. 168-178, 1956.
- [6] BIOT, M. A. Theory of Propagation of Elastic Waves in a Fluid-Saturated Porous Solid. II. Higher-Frequency Range. **The Journal of the Acoustical Society of America**, v. 28, n. 2, p. 179-191, 1956.
- [7] REUSS, A. Berechnung der Fließgrenze von Mischkristallen auf Grund der Plastizitätsbedingung für Einkristalle. **ZAMM - Journal of Applied Mathematics and Mechanics**, v. 9, n. 1, p. 49-58, 1929.
- [8] VOIGT, W. Ueber die Beziehung zwischen den beiden Elasticitätsconstanten isotroper Körper. **Annalen der Physik**, v. 274, n. 12, p. 573-587, 1889.
- [9] STROUSTRUP, B. **The C++ Programming Language**. 4th ed. Addison-Wesley Professional, 2013.
- [10] MEYERS, S. **Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14**. 1st ed. O'Reilly Media, 2014.
- [11] JANERT, P. K. **Gnuplot in Action: Understanding Data with Graphs**. 2nd ed. Manning Publications, 2016.
- [12] WILLIAMS, T.; KELLEY, C. **Gnuplot 6.1 Documentation**. Disponível em: <<http://www.gnuplot.info/documentation.html>>. Acesso em: 28 set. 2025.

Apêndice A

Modelagem Matemática e Algoritmos de Cálculo

Descreve-se neste apêndice o detalhamento dos modelos matemáticos utilizados para a estimativa das propriedades elásticas efetivas do compósito rochoso, bem como o algoritmo central que implementa estes cálculos no sistema desenvolvido. Este material, embora fundamental para a reprodutibilidade e compreensão profunda dos resultados, é apresentado aqui por seu elevado nível de detalhe técnico.

A.1 Modelos de Média Poroelástica

Para estimar as propriedades elásticas de uma rocha (um material compósito), a partir das propriedades de seus minerais constituintes, foram empregados os modelos de média de Voigt, Reuss e Hill. Estes modelos fornecem, respectivamente, os limites superior e inferior, e uma média empírica para os módulos elásticos efetivos.

Dada uma rocha composta por N minerais, onde cada mineral i possui uma fração volumétrica f_i , um módulo de compressibilidade K_i e um módulo de cisalhamento G_i , os módulos efetivos do compósito (K_{eff} , G_{eff}) são calculados da seguinte forma:

Média de Voigt (Limite Superior) Este modelo assume que todos os constituintes sofrem a mesma deformação.

$$K_{Voigt} = \sum_{i=1}^N f_i K_i \quad (A.1)$$

$$G_{Voigt} = \sum_{i=1}^N f_i G_i \quad (A.2)$$

Média de Reuss (Limite Inferior) Este modelo assume que todos os constituintes estão sujeitos à mesma tensão.

$$K_{Reuss} = \left(\sum_{i=1}^N \frac{f_i}{K_i} \right)^{-1} \quad (A.3)$$

$$G_{Reuss} = \left(\sum_{i=1}^N \frac{f_i}{G_i} \right)^{-1} \quad (\text{A.4})$$

Média de Voigt-Reuss-Hill Considerada uma aproximação empírica mais realista, é a média aritmética simples dos limites de Voigt e Reuss.

$$K_{Hill} = \frac{K_{Voigt} + K_{Reuss}}{2} \quad (\text{A.5})$$

$$G_{Hill} = \frac{G_{Voigt} + G_{Reuss}}{2} \quad (\text{A.6})$$

A.2 Algoritmo de Cálculo dos Módulos Elásticos

O núcleo de processamento do sistema, encapsulado na classe `CElasticCalculator`, segue um algoritmo bem definido para determinar as propriedades de uma amostra. O processo pode ser descrito em alto nível pelo seguinte pseudocódigo:

```

1 FUNCAO Calculate(amostra):
2     // 1. Calcula os modulos de compressibilidade (K)
3     k_voigt = m_bulkCalc.CalculateVoigt(amostra)
4     k_reuss = m_bulkCalc.CalculateReuss(amostra)
5     k_hill = (k_voigt + k_reuss) / 2.0
6
7     // 2. Calcula os modulos de cisalhamento (G)
8     g_voigt = m_shearCalc.CalculateVoigt(amostra)
9     g_reuss = m_shearCalc.CalculateReuss(amostra)
10    g_hill = (g_voigt + g_reuss) / 2.0
11
12    // 3. Agrega e retorna os resultados
13    RETORNE um objeto Result contendo:
14        k_voigt, k_reuss, k_hill,
15        g_voigt, g_reuss, g_hill
16 FIM FUNCAO

```

Este algoritmo demonstra a delegação de responsabilidades no sistema: a classe `CElasticCalculator` orquestra o cálculo, enquanto as classes `CBulkCalculator` e `CShearCalculator` realizam os cálculos específicos para cada módulo, acessando a base de dados de minerais conforme a necessidade.

Apêndice B

Código-Fonte Selecionado

Este apêndice apresenta o código-fonte das interfaces (arquivos de cabeçalho `.h`) das principais classes desenvolvidas no projeto. A escolha por apresentar as interfaces visa a oferecer uma visão clara e concisa da arquitetura orientada a objeto, das responsabilidades de cada classe e de suas inter-relações, sem o detalhamento excessivo dos arquivos de implementação.

B.1 Classe CSimulador

Responsável pela orquestração geral do sistema, gerenciando o fluxo de execução e a interação entre os componentes.

```
1 #ifndef CPECAPPLICATION_H
2 #define CPECAPPLICATION_H
3
4 #include "CInput.h"
5 #include "CElasticCalculator.h"
6 #include "CMineralDatabase.h"
7 #include "CReservoirData.h"
8 #include "CPlotManager.h"
9
10 class CSimulador {
11 public:
12     CSimulador();
13     void Run();
14
15 private:
16     CInput m_inputManager;
17     CMineralDatabase m_database;
18     CElasticCalculator m_calculator;
19     CReservoirData m_reservoirData;
20     CPlotManager m_plotManager;
21     bool m_running;
22
23     void Menu();
24     void Initialize();
```

```

25     void Finalize();
26 };
27
28 #endif

```

B.2 Classe CElasticCalculator

Encapsula a lógica de cálculo dos módulos elásticos, utilizando composição para delegar os cálculos específicos.

```

1  #ifndef CELASTICCALCULATOR_H
2  #define CELASTICCALCULATOR_H
3
4  #include "CMineralDatabase.h"
5  #include "CAmostra.h"
6  #include "CBulkCalculator.h"
7  #include "CShearCalculator.h"
8
9  class CElasticCalculator {
10 public:
11     struct Result {
12         float bulkVoigt, bulkReuss, bulkHill;
13         float shearVoigt, shearReuss, shearHill;
14     };
15
16     explicit CElasticCalculator(const CMineralDatabase& db);
17     Result Calculate(const CAmostra& amostra) const;
18
19 private:
20     CBulkCalculator m_bulkCalc;
21     CShearCalculator m_shearCalc;
22 };
23
24 #endif // CELASTICCALCULATOR_H

```

B.3 Classe CAmostra

Representa a entidade de dados de uma amostra de rocha, encapsulando todas as suas propriedades.

```

1  #ifndef CAMOSTRA_H
2  #define CAMOSTRA_H
3
4  #include <string>
5  #include <map>
6
7  class CAmostra {
8 public:

```

```
9     CAmostra();
10     CAmostra(const std::string& reservatorio, const std::string& nome,
11             float profundidade, float porosidade,
12             const std::map<std::string, float>& minerais);
13
14     const std::string& NomeReservatorio() const;
15     const std::string& NomeAmostra() const;
16     float Profundidade() const;
17     float Porosidade() const;
18     const std::map<std::string, float>& Minerais() const;
19
20 private:
21     std::string m_reservatorio;
22     std::string m_nome;
23     float m_profundidade;
24     float m_porosidade;
25     std::map<std::string, float> m_minerais;
26 };
27
28 #endif
```

Índice Remissivo

Análise orientada a objeto, 13

AOO, 13

Casos de uso, 8

colaboração, 19

comunicação, 19

Concepção, 6

Diagrama de colaboração, 19

Diagrama de componentes, 27

Diagrama de estado, 20

Diagrama de execução, 29

Diagrama de implantação, 29

Diagrama de sequência, 18

Elaboração, 10

Especificação, 7

especificação, 7

estado, 20

Eventos, 18

Mensagens, 18

Plataformas, 24

POO, 25

Projeto do sistema, 23

Projeto orientado a objeto, 25

Protocolos, 23

Recursos, 24

Requisitos, 7

Requisitos funcionais, 7

Requisitos não funcionais, 8